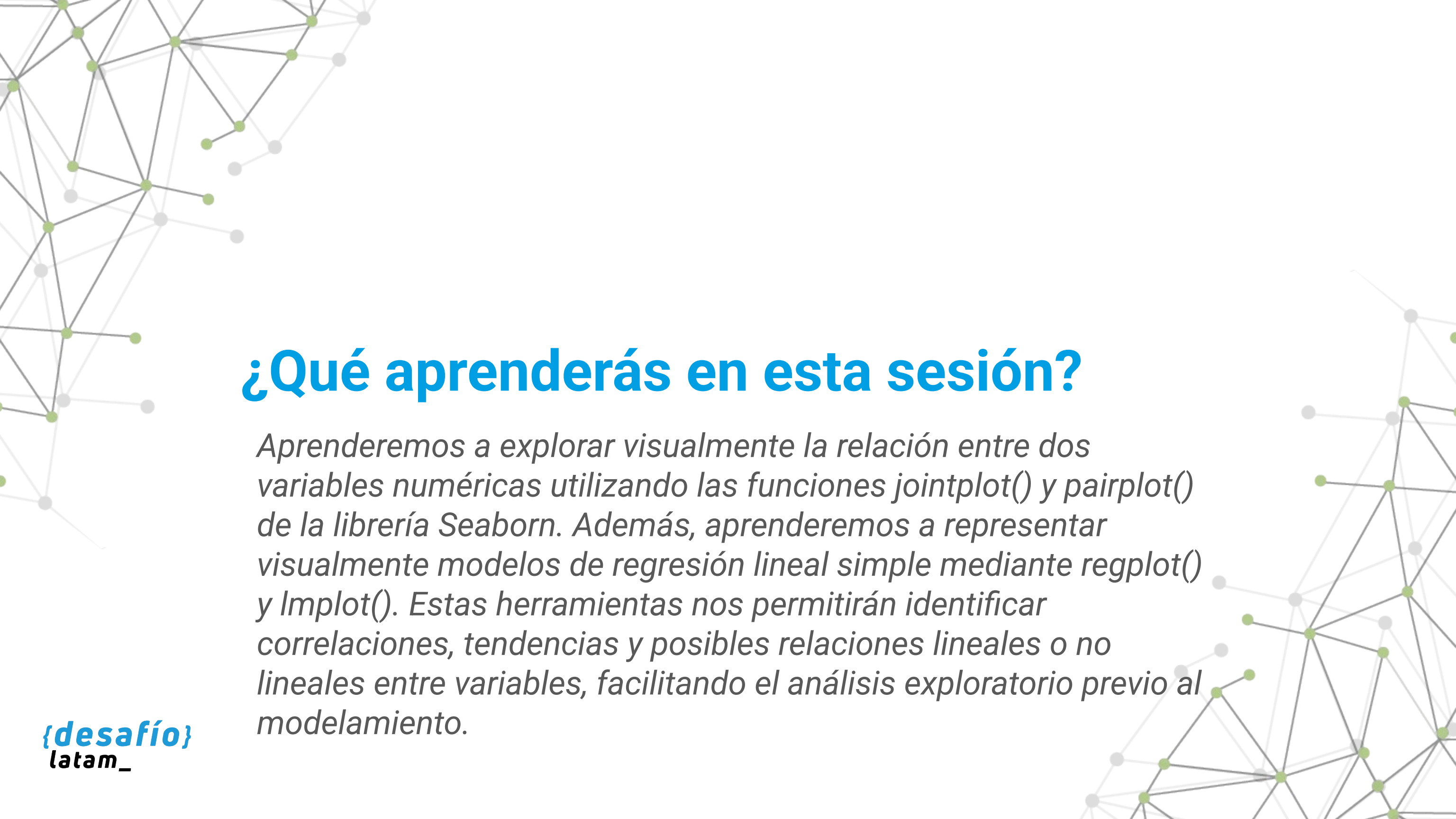




Visualización de datos

Clase 3 - Gráficos de relaciones bivariadas y regresiones con Seaborn



¿Qué aprenderás en esta sesión?

Aprenderemos a explorar visualmente la relación entre dos variables numéricas utilizando las funciones `jointplot()` y `pairplot()` de la librería Seaborn. Además, aprenderemos a representar visualmente modelos de regresión lineal simple mediante `regplot()` y `lmlplot()`. Estas herramientas nos permitirán identificar correlaciones, tendencias y posibles relaciones lineales o no lineales entre variables, facilitando el análisis exploratorio previo al modelamiento.

Desafío inicial

Una empresa de tecnología quiere entender mejor la relación entre el tiempo que las personas usuarias pasan en su plataforma y la cantidad de compras realizadas. El equipo de análisis desea comprobar si existe una correlación entre ambas variables y, si es posible, trazar una línea de tendencia para modelar la relación.

¿Cómo podemos visualizar correctamente la relación entre dos variables numéricas y representar, si existe, un patrón lineal o de dependencia?

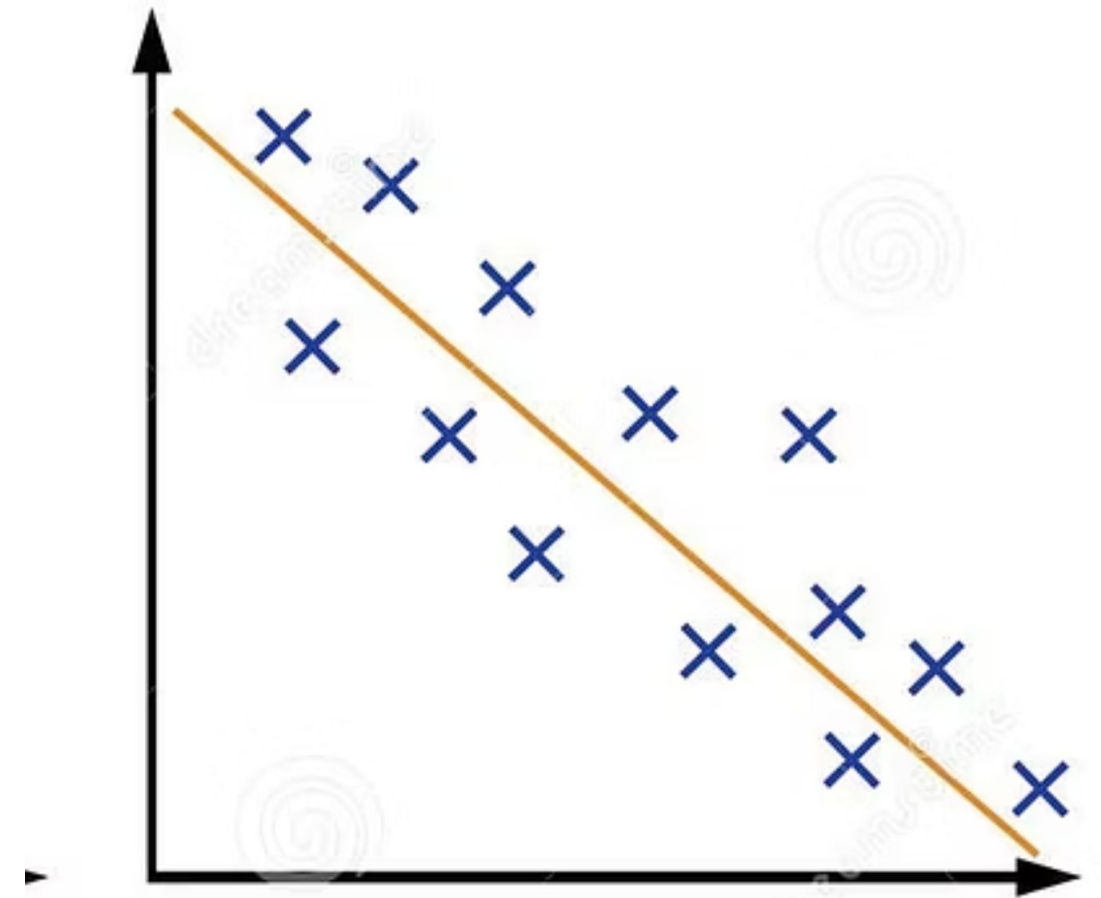
Para responder esta interrogante, será necesario visualizar la relación entre ambas variables mediante gráficos de dispersión bivariados. En esta sesión aprenderemos a utilizar `jointplot()` y `pairplot()` para detectar correlaciones visuales, y luego aplicaremos `regplot()` y `lmplot()` para incluir líneas de regresión que nos permitan explorar relaciones lineales entre estas variables.

/* Gráficos de dispersión */

Gráficos de dispersión bivariados: `jointplot()` y `pairplot()`

¿Puedes identificar visualmente si dos variables se relacionan de forma positiva o negativa, o si no hay relación alguna?

Los gráficos de dispersión son esenciales para explorar relaciones entre dos variables numéricas. En visualización de datos, permiten identificar correlaciones, grupos, outliers o patrones de dependencia. Seaborn ofrece herramientas poderosas para este propósito, destacando `jointplot()` y `pairplot()`.

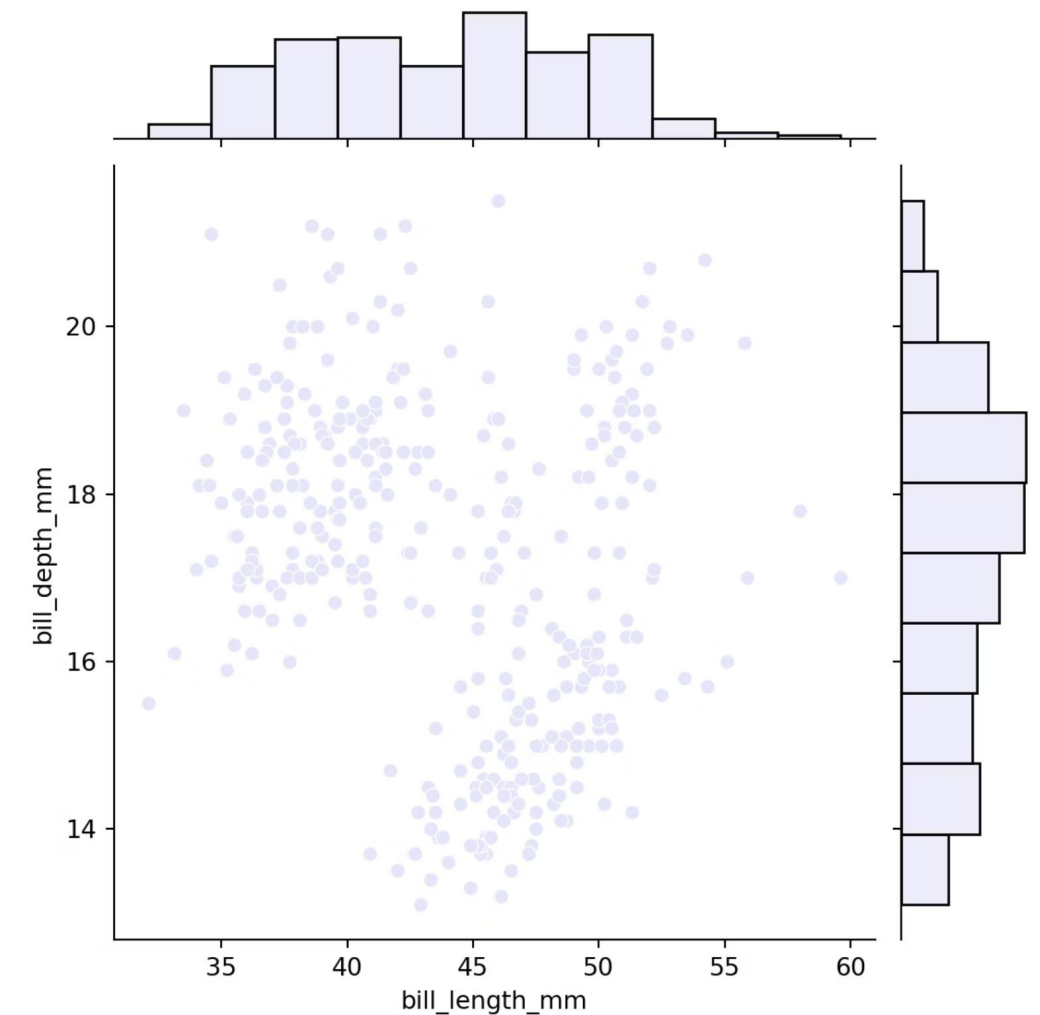


**Negative
Correlation**

jointplot()

jointplot() combina un gráfico de dispersión con histogramas marginales para mostrar la distribución individual de cada variable. Es útil para explorar relaciones bivariadas y, al mismo tiempo, obtener una idea de la distribución de cada variable por separado.

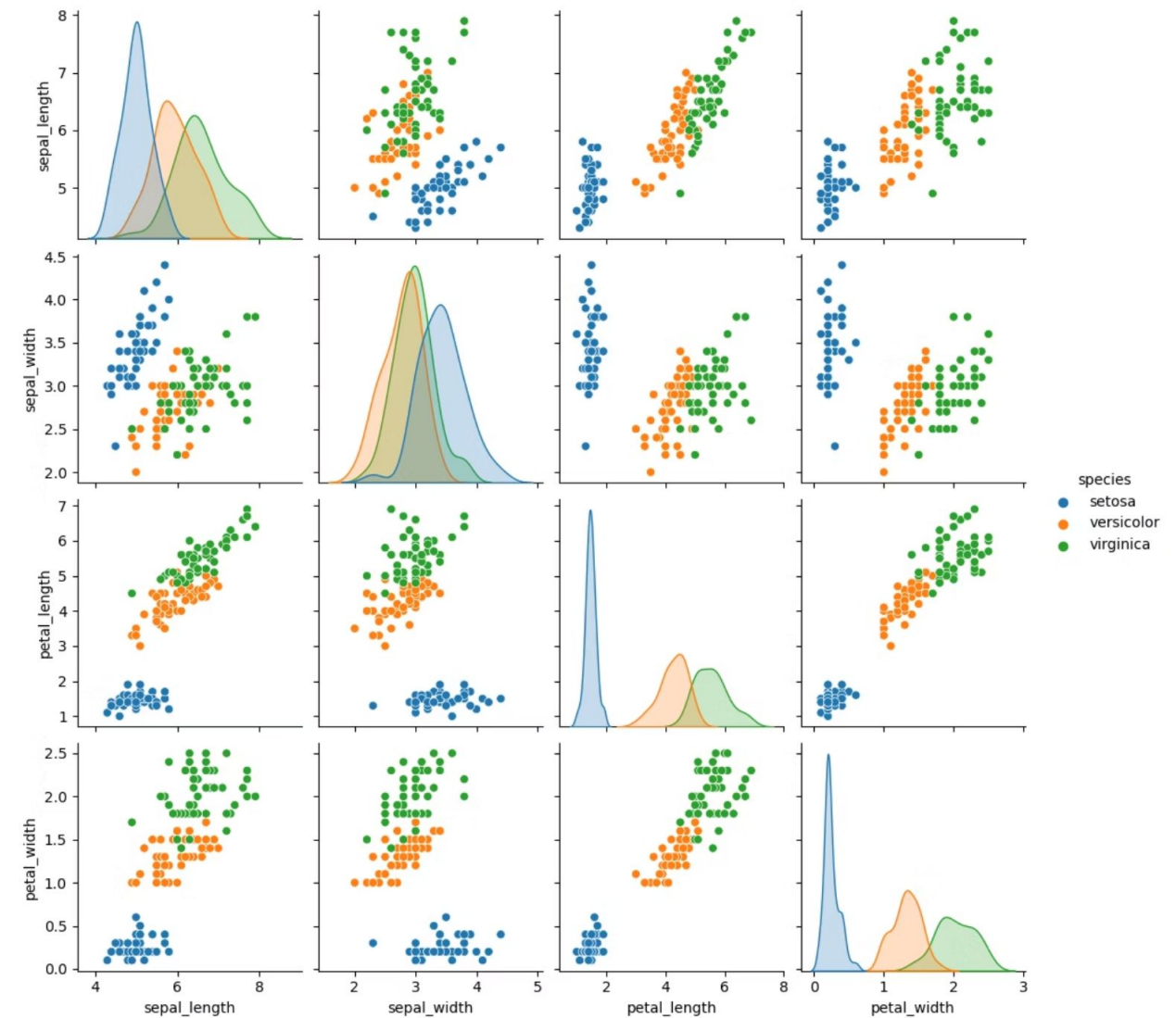
```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
# Cargar datos de ejemplo
clientes =
pd.read_csv("comportamiento_usuarios_streaming.csv")
# Gráfico de dispersión con histogramas marginales
sns.jointplot(data=clientes, x="min_conexion_dia",
y="modulos_completados", kind="scatter")
plt.show()
# También puede incluir una regresión
sns.pairplot(data=clientes[['min_conexion_dia',
'modulos_completados', 'sesiones_dia']], diag_kind='kde')
```



pairplot()

pairplot() permite visualizar todas las combinaciones posibles de variables numéricas en pares de gráficos de dispersión. Es ideal cuando queremos evaluar múltiples relaciones bivariadas al mismo tiempo.

```
sns.pairplot(data=clientes[['min_conexion_dia', 'modulos_completados', 'sesiones_dia']],  
diag_kind='kde')
```

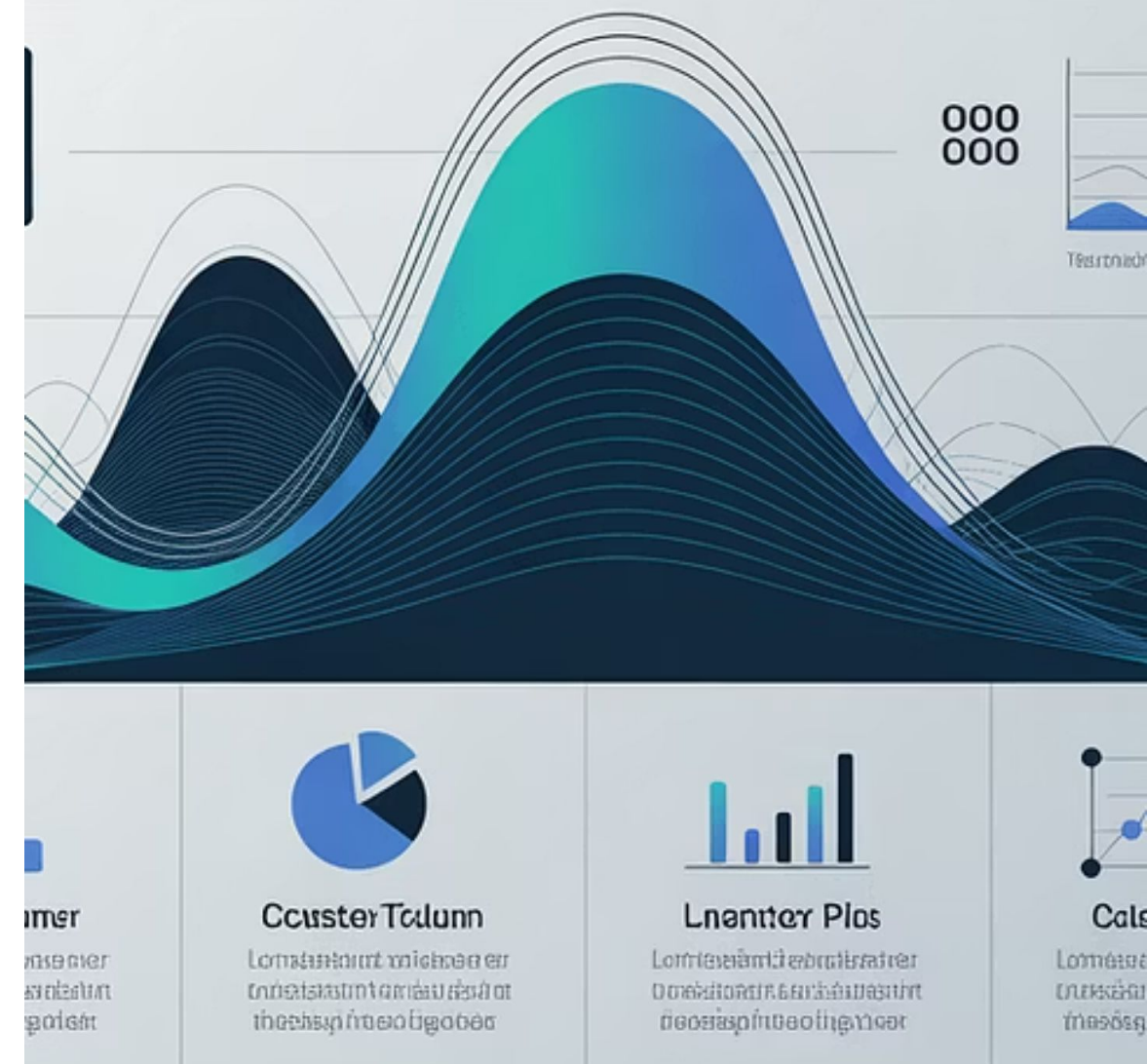


Buenas prácticas con gráficos bivariados

Antes de usar `jointplot()` o `pairplot()`, es recomendable tener en cuenta lo siguiente:

- Elige variables numéricas limpias y sin valores extremos dominantes.
- Usa `kind="kde"` si prefieres visualizar densidades en vez de histogramas.
- Limita la cantidad de variables en `pairplot()` a 4 o 5 para evitar sobrecarga visual.
- Configura `height` y `aspect` en `jointplot()` para controlar tamaño y proporciones.
- Siempre etiqueta los ejes y considera una paleta de colores accesible.

DATA VISUALIZATION BEST PRACTICES



Errores comunes

Aunque las funciones `jointplot()` y `pairplot()` de Seaborn facilitan una exploración visual poderosa de relaciones entre variables, su mal uso puede llevar a interpretaciones incorrectas o visualizaciones poco útiles. Muchos de estos errores provienen de una comprensión incompleta de las características de las variables o de un uso inadecuado de los parámetros disponibles. A continuación, se describen algunos de los errores más frecuentes que deben evitarse al trabajar con gráficos bivariados:



Usar variables categóricas como si fueran numéricas

Esto genera gráficos sin sentido que no representan relaciones reales.



Sobrecargar `pairplot()` con demasiadas columnas

Reduce la legibilidad y hace difícil interpretar los patrones.



No interpretar correctamente escalas en ejes

Por ejemplo, unidades de tiempo que pueden llevar a conclusiones erróneas.



Olvidar activar `plt.show()` al usar solo `matplotlib`

Resulta en visualizaciones que no se muestran correctamente.



Ignorar la distribución marginal

Que puede explicar patrones importantes en la dispersión.

¿Cuál herramienta de dispersión usar?

Al enfrentarnos a la necesidad de visualizar relaciones entre variables numéricas, es común preguntarse cuál función utilizar. Tanto `jointplot()` como `pairplot()` permiten observar asociaciones entre variables, pero cada una responde a propósitos distintos. La siguiente tabla comparativa te ayudará a elegir la herramienta más adecuada según tus necesidades analíticas:

Comparación de herramientas

Característica	jointplot()	pairplot()
Relación	Bivariada (una pareja de variables)	Multivariada (varias combinaciones de pares)
Visualizaciones marginales	Sí (histograma, KDE, etc.)	Sí (en la diagonal: histograma o KDE)
Personalización detallada	Más control con height, ratio, space	Menos granular, pero flexible con diag_kind, hue
Ideal para...	Análisis profundo de una relación específica	Exploración global de relaciones múltiples
Rendimiento con muchos datos	Rápido	Más lento si hay muchas columnas

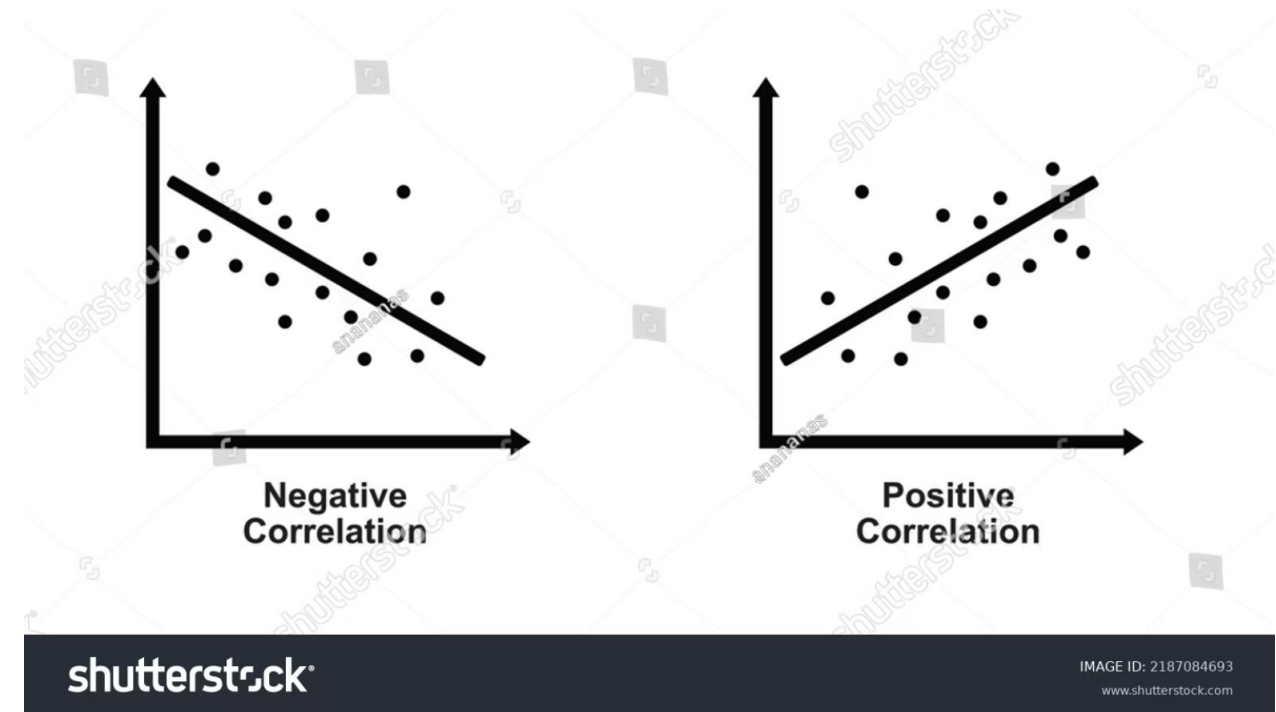
Recomendación: Usa jointplot() si ya tienes un par de variables objetivo. Prefiere pairplot() cuando estás en una fase exploratoria inicial.

Visualización de regresiones con regplot() y Implot()

Una vez que identificamos relaciones visuales entre variables, el siguiente paso es cuantificarlas o modelarlas. Para ello, podemos agregar una línea de tendencia con regresión. Pasemos a explorar cómo hacerlo visualmente con regplot() y Implot().

¿Cómo se ve una tendencia lineal en un gráfico y cómo puedes saber si una relación es positiva o negativa?

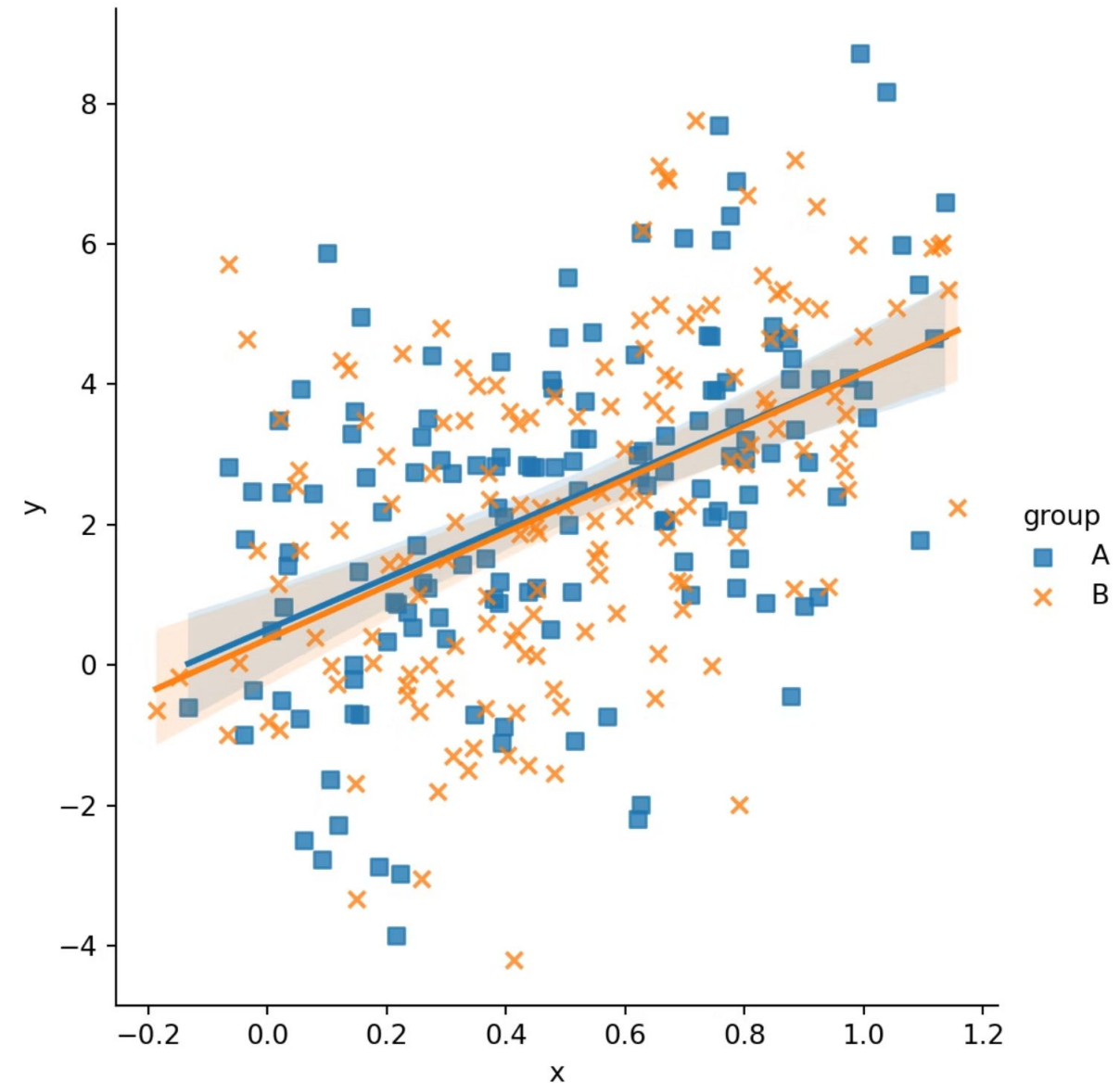
Al detectar relaciones visuales entre variables, muchas veces queremos entender si existe una relación lineal (es decir, si a mayor valor en X también aumenta o disminuye Y). Seaborn ofrece funciones específicas para superponer líneas de regresión y observar visualmente el ajuste. Esto facilita análisis exploratorios previos a aplicar modelos más formales.



regplot()

Esta función permite construir un gráfico de dispersión y superponer una recta de regresión lineal automáticamente. Ideal para visualizar tendencias lineales simples.

```
sns.regplot(data=clientes,  
x="min_conexion_dia", y="modulos_completados")  
plt.title("Relación entre tiempo de conexión  
y módulos completados")  
plt.show()  
# Puedes ajustar elementos del modelo:  
sns.regplot(data=clientes,  
x="min_conexion_dia", y="modulos_completados",  
ci=95, scatter_kws={"color": "gray"},  
line_kws={"color": "red"})
```



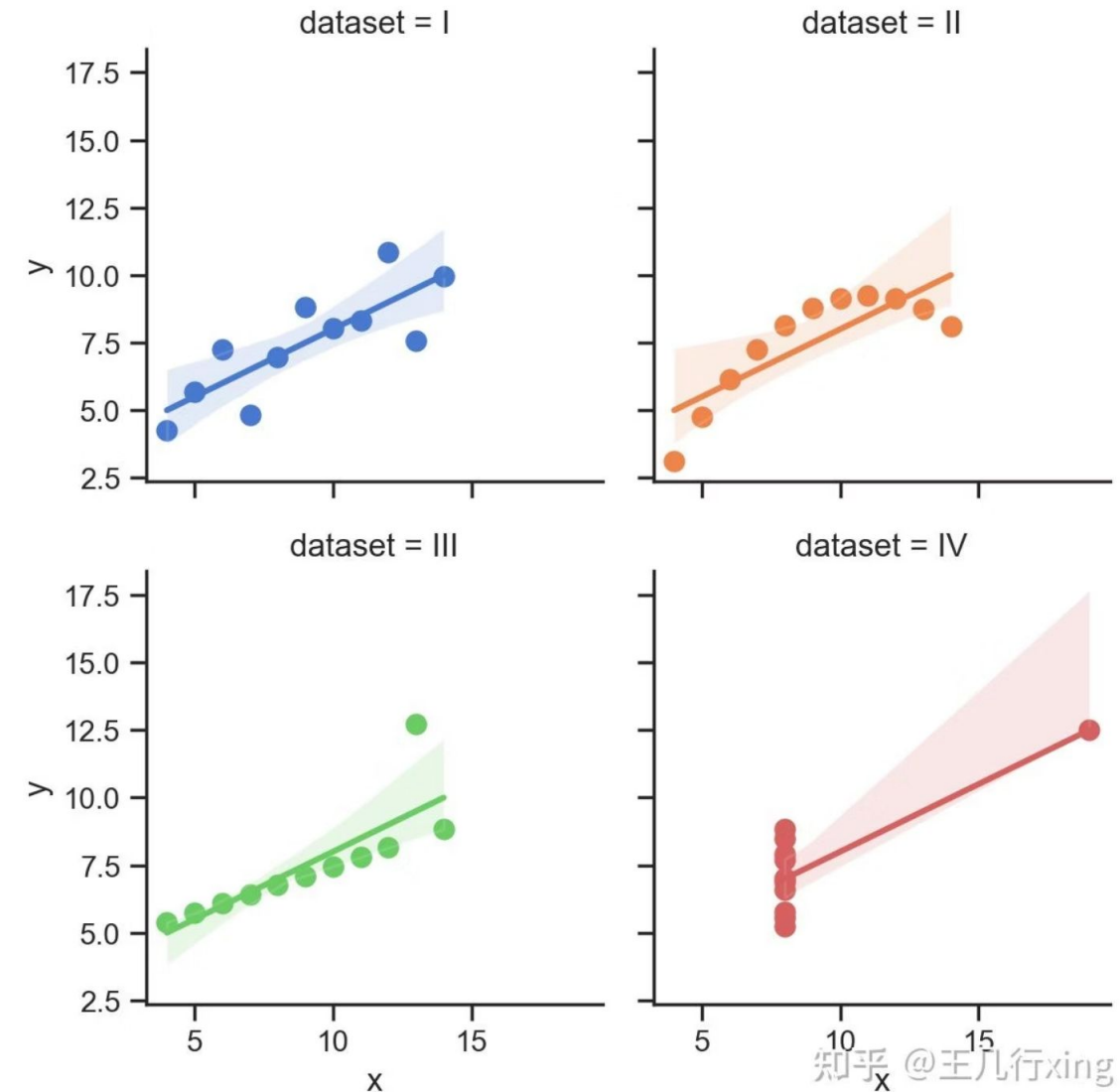
Implot()

Implot() extiende la funcionalidad de regplot() para trabajar con múltiples subconjuntos de datos o incorporar variables categóricas. Internamente, usa regplot() pero permite crear facetas y aplicar modelos por grupo.

```
sns.lmplot(data=clientes,  
x="min_conexion_dia",  
y="modulos_completados",  
col="categoria_usuario",  
hue="categoria_usuario", height=4, aspect=1)
```

Esto genera una regresión por categoría (por ejemplo, personas nuevas vs. recurrentes).

[31]: <seaborn.axisgrid.FacetGrid at 0x1712ef6a0>



Buenas prácticas al visualizar regresiones

Visualizar regresiones no se trata solo de trazar una línea recta sobre un gráfico de dispersión. Para que la interpretación sea válida y útil, es fundamental que existan condiciones adecuadas, como la linealidad de la relación entre variables o la correcta segmentación por subgrupos. Además, la personalización de los elementos visuales ayuda a comunicar con claridad los hallazgos, lo que resulta clave en entornos profesionales donde las visualizaciones son insumos para la toma de decisiones.

Antes de aplicar visualizaciones de regresión:

- Asegúrate de que la relación entre variables parezca lineal.
- Utiliza `ci` para visualizar intervalos de confianza.
- Usa `scatter_kws` y `line_kws` para personalizar colores y estilos.
- Evita superponer demasiadas regresiones en un mismo gráfico.
- Usa `Implot` para segmentar por grupos solo si hay diferencias significativas.

Errores comunes

El uso incorrecto de gráficos de regresión puede generar conclusiones engañosas. Muchas veces se aplican líneas de ajuste en contextos donde no corresponde, o se malinterpreta la visualización como una prueba de causalidad. Estos errores comprometen la calidad del análisis exploratorio y pueden llevar a decisiones erradas en procesos productivos o estrategias de negocio. A continuación, se presentan algunos errores que deben evitarse al emplear este tipo de gráficos.



Usar regresión con variables fuertemente no lineales

Genera líneas de tendencia que no representan adecuadamente la relación real.



No filtrar outliers que distorsionan la recta de ajuste

Puede llevar a conclusiones erróneas sobre la tendencia general.



Interpretar como causal una relación solo visual

Correlación no implica causalidad, un error fundamental en análisis de datos.



Omitir leyendas o etiquetas necesarias

Dificulta la interpretación correcta del gráfico por parte de la audiencia.



Confundir correlación visual con validez estadística

Una línea que parece ajustarse bien visualmente no garantiza significancia estadística.

Parámetros útiles en regresiones visuales

Una buena visualización no solo depende de los datos, sino también del conocimiento de los parámetros que permiten ajustar el gráfico para que sea informativo, claro y profesionalmente útil. Tanto `regplot()` como `lmplot()` en Seaborn ofrecen diversas opciones que permiten personalizar desde la apariencia del gráfico hasta los cálculos estadísticos subyacentes.

A continuación, presentamos una tabla con algunos de los parámetros más relevantes que puedes utilizar para mejorar tus visualizaciones de regresión:

ession Parameters



Parámetros de regresión

Parámetro	Función	Descripción breve
ci	Ambas	Intervalo de confianza (por defecto 95%). Se puede desactivar con ci=None.
scatter_kws	Ambas	Diccionario para personalizar los puntos (color, tamaño, opacidad).
line_kws	Ambas	Diccionario para personalizar la línea de regresión (color, estilo, ancho).
hue	Implot	Permite colorear por categoría. Muy útil para segmentar relaciones.
col / row	Implot	Crea subgráficos (facetas) por categoría, permitiendo comparación visual directa.

Interpretación de relaciones gráficas

Consejo

Asegúrate de que los colores y estilos usados en líneas y puntos no generen confusión. Considera accesibilidad y claridad en contextos de presentación.

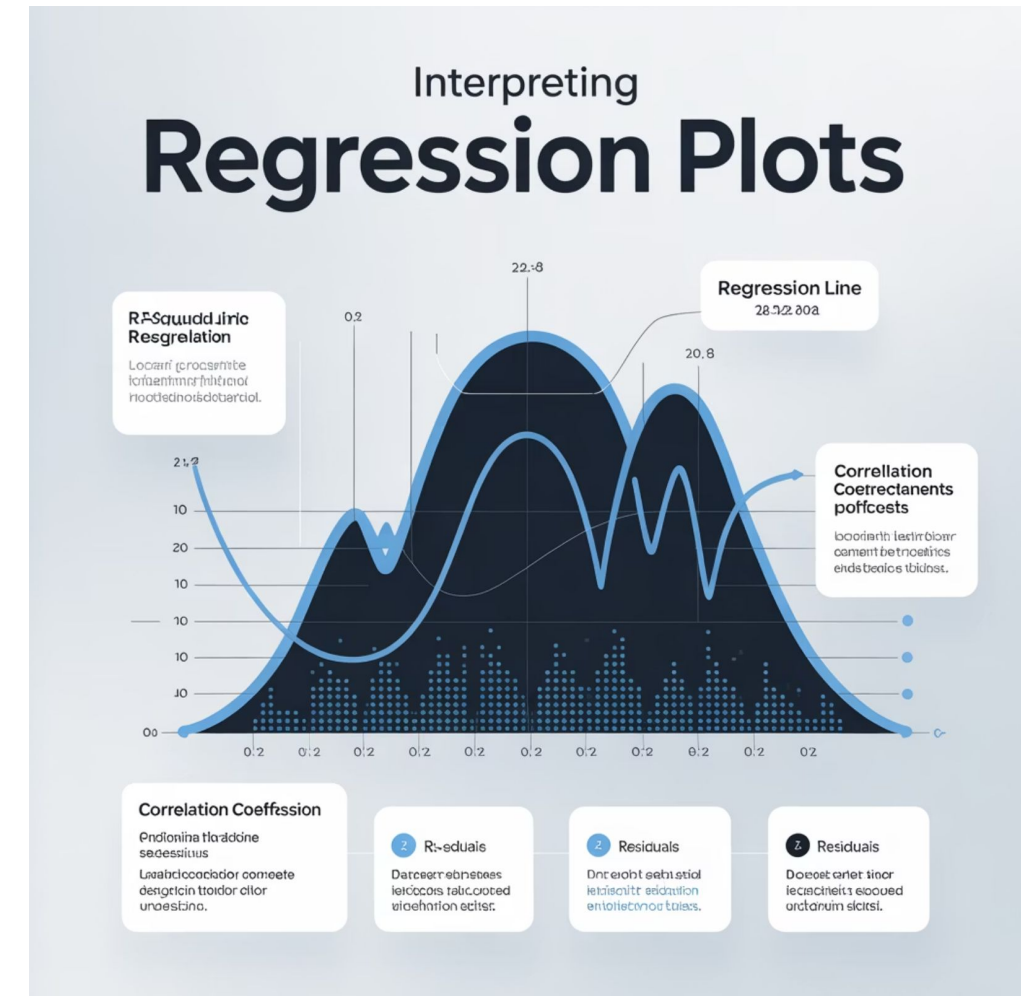
¿Cómo interpretar gráficamente relaciones?

Un gráfico bien hecho comunica más que una tabla, pero solo si sabemos qué mirar. Aquí algunas señales gráficas claves:

- Pendiente positiva o negativa: Indica tipo de relación.
- Densidad de puntos cerca de la línea: Sugiere buen ajuste.
- Distribución en los márgenes: Muestra si hay asimetrías o multimodalidades.
- Puntos alejados o outliers: Pueden distorsionar la interpretación si no se abordan.

Esto ayuda a cerrar con una reflexión crítica: una buena visualización no solo ilustra, sino que también cuestiona.

Las visualizaciones con líneas de regresión permiten no solo observar tendencias, sino también comunicar de manera clara relaciones entre variables. Son una herramienta fundamental para el análisis exploratorio antes de construir modelos predictivos más robustos.



Actividad guiada: Explorando relaciones y tendencias con gráficos bivariados



Ejercicio

Explorando relaciones y tendencias con gráficos bivariados

Objetivo de la actividad

Aplicar técnicas de visualización bivariada utilizando Seaborn para explorar relaciones entre variables numéricas, identificar patrones conjuntos y representar tendencias mediante líneas de regresión.

Situación problema

El equipo de análisis de una plataforma de educación online busca identificar patrones que expliquen por qué ciertas personas usuarias completan más módulos que otras. Sospechan que el tiempo de conexión diaria puede ser un buen predictor, pero necesitan una exploración visual que les permita observar si esta relación existe y si es lineal.

Tu tarea como parte del equipo será graficar y analizar visualmente estas variables para facilitar una decisión basada en evidencia.



Información a utilizar

```
usuario_id, categoria_usuario, min_conexion_dia,  
modulos_completados, sesiones_dia  
U001, nuevo, 32, 2, 1  
U002, recurrente, 75, 6, 3  
U003, nuevo, 15, 1, 1  
U004, recurrente, 90, 7, 4  
U005, nuevo, 45, 3, 2  
U006, recurrente, 60, 5, 3  
U007, nuevo, 25, 2, 1  
U008, recurrente, 80, 6, 4  
U009, nuevo, 10, 1, 1  
U010, recurrente, 100, 8, 5
```

Ejercicio

Instrucciones

En esta actividad deberás analizar visualmente la relación entre el tiempo de conexión y el avance académico en una plataforma de aprendizaje en línea. Para ello, sigue los pasos:

Cargar librerías y datos

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
clientes =
pd.read_csv("comportamiento_usuarios_streaming.csv")
```

{desafío}
latam_



Ejercicio

Instrucciones

Usar `jointplot()` para explorar la relación con histogramas marginales

```
sns.jointplot(data=clientes,  
x="min_conexion_dia",  
y="modulos_completados", kind="scatter")  
plt.show()
```

Aplicar `pairplot()` si se quieren ver múltiples relaciones entre variables

```
sns.pairplot(data=clientes[['min_conexion_dia',  
'modulos_completados', 'sesiones_dia']],  
diag_kind="kde")
```



Ejercicio

Instrucciones

Visualizar la tendencia lineal con regplot()

```
sns.regplot(data=clientes,  
x="min_conexion_dia",  
y="modulos_completados", ci=95)  
plt.title("Relación entre tiempo de  
conexión y módulos completados")  
plt.show()
```

Comparar tendencias por categoría con lmpplot()

```
sns.lmpplot(data=clientes,  
x="min_conexion_dia",  
y="modulos_completados",  
hue="categoria_usuario")
```



Resumen

1

Comprendimos el valor de los gráficos de dispersión para explorar visualmente relaciones entre dos variables numéricas.

2

Aprendimos a utilizar `jointplot()` para visualizar relaciones bivariadas junto con la distribución de cada variable.

3

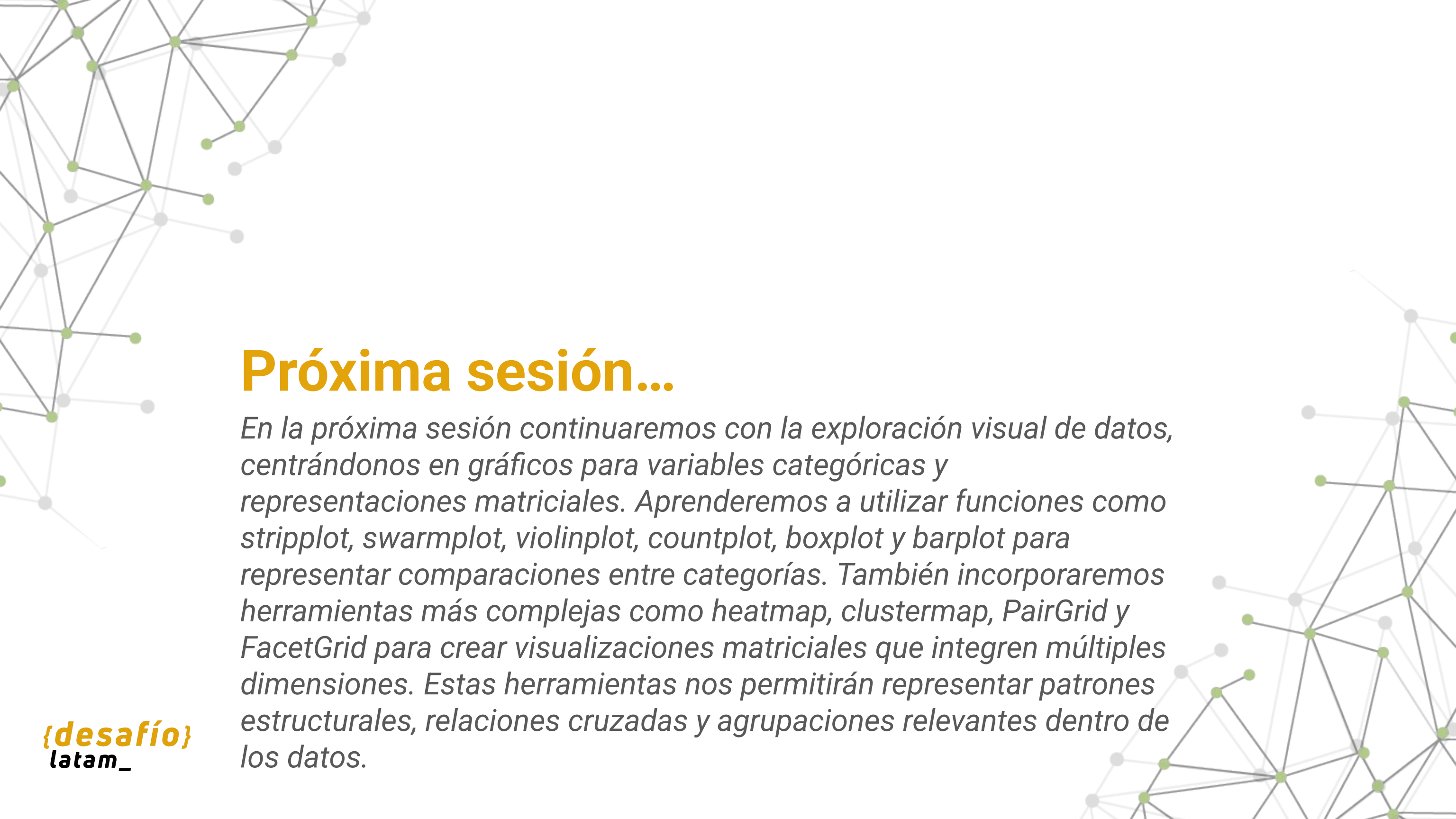
Utilizamos `pairplot()` para explorar múltiples relaciones entre variables de forma simultánea.

4

Aplicamos `regplot()` y `lmlplot()` para representar tendencias lineales y compararlas entre grupos.

5

Identificamos buenas prácticas y errores comunes en el uso de estos gráficos para mejorar la interpretación visual.



Próxima sesión...

En la próxima sesión continuaremos con la exploración visual de datos, centrándonos en gráficos para variables categóricas y representaciones matriciales. Aprenderemos a utilizar funciones como stripplot, swarmplot, violinplot, countplot, boxplot y barplot para representar comparaciones entre categorías. También incorporaremos herramientas más complejas como heatmap, clustermap, PairGrid y FacetGrid para crear visualizaciones matriciales que integren múltiples dimensiones. Estas herramientas nos permitirán representar patrones estructurales, relaciones cruzadas y agrupaciones relevantes dentro de los datos.